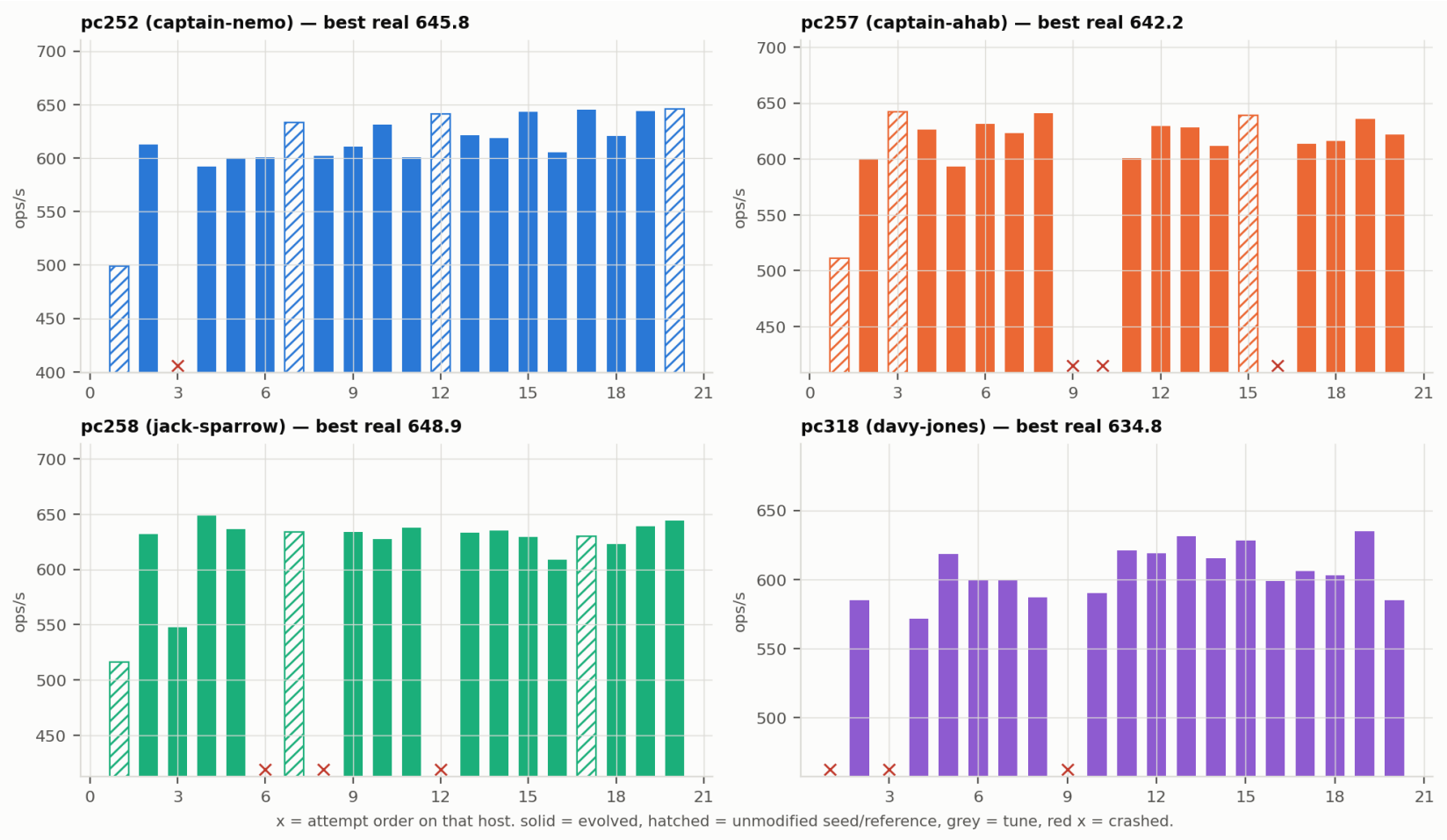
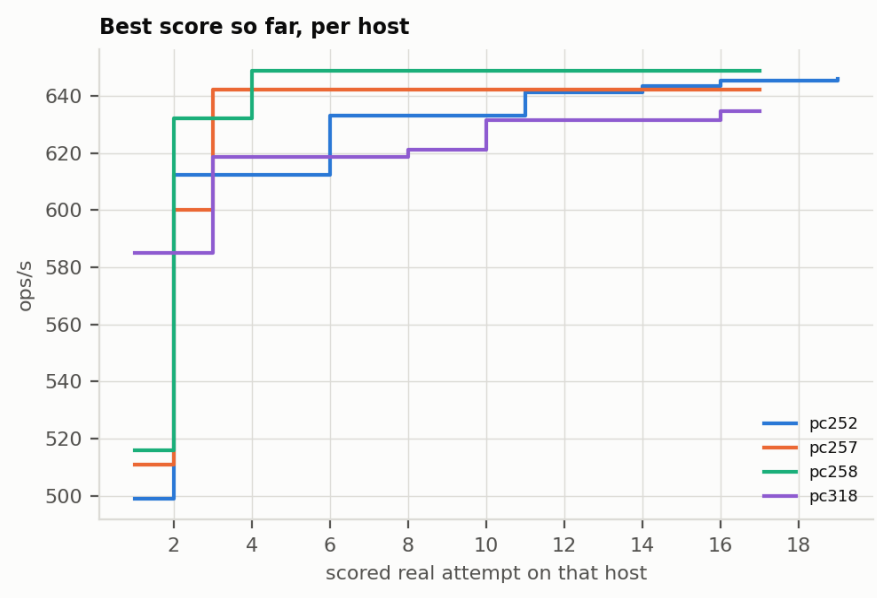
Multi-host CORAL evolution — run 2026-10-26_235915

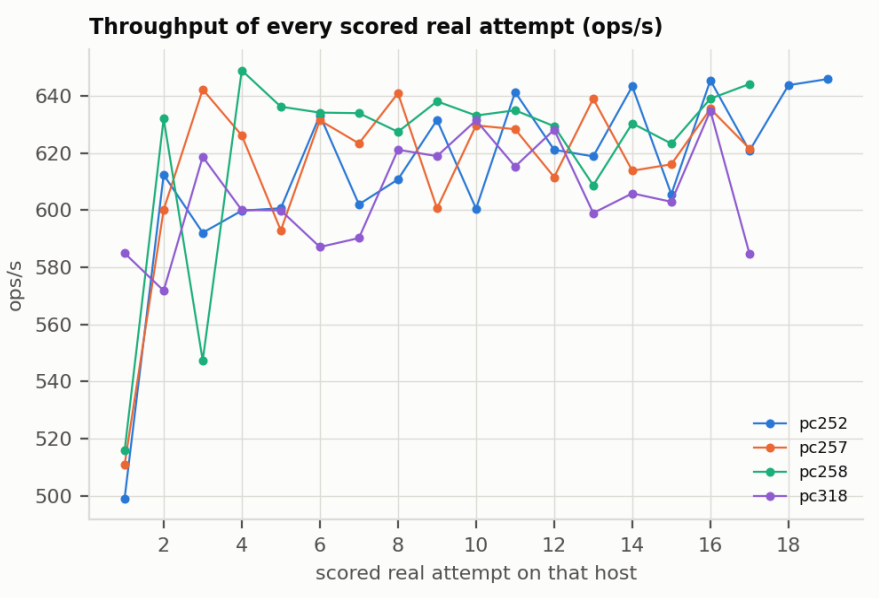
4 agents, one per CloudLab host (each agent evaluated only on its own host, in parallel); score = absolute measured throughput (ops/s). Budget 20 real evaluations per host; all attempts are in one shared leaderboard.

host	agent	real / tune / crashed	best score	its tput (ops/s)	max tput (ops/s)	host no-op baseline (ops/s, n=8; tanh only)	LLM cost
pc252	captain-nemo	20 / 0 / 1	#80 486071a8 645.8	645.8	645.8	-	$7
pc257	captain-ahab	20 / 0 / 3	#10 069942bd 642.2	642.2	642.2	-	$6
pc258	jack-sparrow	20 / 0 / 3	#15 b65fa44f 648.9	648.9	648.9	-	$7
pc318	davy-jones	20 / 0 / 3	#67 215802ea 634.8	634.8	634.8	-	$7

- Overall best: pc258 (jack-sparrow), #15 b65fa44f, score 648.9, 648.9 ops/s — agent-evolved.
- Run stop: max_real_attempts; wall time 1.06 h; total LLM cost $27; gateway requests {400: 140, 200: 594}.
- Hosts differ in speed, so raw throughput is not comparable across hosts: compare changes within a host, not levels across hosts.







Top 10 attempts across all hosts (real, scored)

#	host	agent	attempt	commit	score	tput	MiB read	refaults	design / title
1	pc258	jack-sparrow	15	b65fa44f	648.9	648.9	163	16831	v3: scan-first LRU + probation list. New non-scan folios go to prob_list; first non-scan access promotes to main LRU. Evict order
2	pc252	captain-nemo	80	486071a8	645.8	645.8	180	20641	final eval: noop run 3 on pc252 (runs 633.2, 641.3) to pin the noop mean against the near-noop hybrid (645.4, 643.7) and no-victim
3	pc252	captain-nemo	77	624a8c66	645.4	645.4	179	20237	dose test: conservative-assist (scan-only victims >1024) rate-limited to 1 of 16 evict calls. Full-rate = 605.6, no-victims = 643.
4	pc258	jack-sparrow	64	510f3bc0	644.1	644.1	162	16485	FINAL (20/20): v3 with SCAN_KEEP 256->32 (last untested scan-reserve point; 2048 was neutral). pc258 summary: v3 mean 638.3 (649,6
5	pc252	captain-nemo	79	bc488f8e	643.7	643.7	178	20102	replicate 2 of rate-1/16 scan-only-victim hybrid (624a8c66: 645.4) on pc252; noop 633/641. n=2 vs n=2 test of whether near-noop be
6	pc252	captain-nemo	75	b3a25ca0	643.3	643.3	180	20442	decomposition test: jack v3 keep32 with full hook list bookkeeping (add/move) but evict_folios proposes NOTHING (kernel does all r
7	pc257	captain-ahab	10	069942bd	642.2	642.2	186	21776	calibration: unmodified noop (kernel default reclaim) on my host pc257, to measure baseline + diagnostic probes
8	pc252	captain-nemo	71	27543df2	641.3	641.3	181	20610	noise replicate: noop on pc252 run 2 (run 1 = 633.2, 13c7f750). Gives pc252 its first identical-code pair for the team's noise est
9	pc257	captain-ahab	26	18525923	640.9	640.9	177	19924	structural 1/3 2Q-CLOCK (on bd194ffb, no scan special-casing): main list becomes freq CLOCK (access freq++ cap 7, no list_move; ev
10	pc258	jack-sparrow	61	1ff6958a	639.0	639.0	160	16148	replication 3 of v3 (b65fa44f: 649, 627). pc258 means: noop 632 (634,630); v6 632 (638,635,623). Testing whether v3 (no metadata,

All attempts on pc252 (captain-nemo)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	ae2fc9b0	real =get_scan	499.2	499.2	134	10673	40	baseline: reference get_scan.c unchanged (scan_pids list + sampled LFU) to calibrate my host
2	44143b75	real	612.4	612.4	164	17116	40	scan-quarantine LRU v1 (structural 1/3): scan_pids folios -> FIFO scan_list evicted first above 256-page reserve; main_list true LRU (move-t
3	90945d6e	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
4	97e2752b	real	592.1	592.1	158	15743	41	SLRU v1 retry (structural 1/3; fix: cache_pages rodata u64->size_t): scan_list FIFO reserve 256 + probation LRU + protected LRU cap 3/4 cach
5	bfd0bfd5	real	599.8	599.8	160	16363	41	ARC-lite v1 (structural 1/3): scan FIFO (reserve 256) + prob (unread readahead) + once LRU + hot LRU (2nd read or ghost hit; cap 3/4 cache);
6	3de110b9	real	600.6	600.6	160	16354	40	file-tail meta list on jack v3 (b65fa44f): last 8 pages of each file (LevelDB filter/index/footer) in own LRU capped 3072 pages, evicted aft
7	13c7f750	real =noop	633.2	633.2	180	20438	41	calibration: unmodified noop (kernel default; jack's 25d5e3f8 + trailing comment) on pc252 — my host lacks a noop reference; tells whether m
8	68d5c093	real	602.0	602.0	165	17225	41	workingset emulation v1 (structural 1/3, new lane): kernel-style inactive/active with two-touch activation (ref bit) + refault-DISTANCE-gate
9	7f817b2f	real	610.7	610.7	159	16077	41	reviewer replication on 3rd host: jack v3 SCAN_KEEP=32 (510f3bc0, 644 on pc258) unchanged on pc252; my noop here = 633. Tests 'v3 family bea
10	961a0e71	real	631.5	631.5	160	16305	41	fallback-share hybrid on jack v3 SCAN_KEEP=32 (510f3bc0; 611 on pc252, noop 633): every 4th evict_folios call returns nothing so kernel defa
11	2805fbba	real	600.5	600.5	164	17075	41	fallback-share hybrid FALLBACK_EVERY=2 (half of evict_folios calls defer to kernel) on jack v3 keep32; FE=4 got 631.5, no-fallback 610.7, no
12	27543df2	real =noop	641.3	641.3	181	20610	41	noise replicate: noop on pc252 run 2 (run 1 = 633.2, 13c7f750). Gives pc252 its first identical-code pair for the team's noise estimate (Q3
13	66a68e93	real	621.1	621.1	159	16032	41	replicate 2 of fallback-1/4 hybrid (961a0e71: 631.5) on pc252; noop here 633/641, v3-no-fallback 611, FE=2 600. Decides whether the hybrid r
14	93689216	real	618.8	618.8	177	19992	41	reviewer replication, 4th host: ahab 2Q-CLOCK (18525923, no scan special-casing; pc257 641/612/622, pc318 621/619/603) unchanged on pc252. n
15	b3a25ca0	real	643.3	643.3	180	20442	41	decomposition test: jack v3 keep32 with full hook list bookkeeping (add/move) but evict_folios proposes NOTHING (kernel does all reclaim). I
16	109c797b	real	605.5	605.5	164	17156	41	conservative-assist hybrid v1: track only scan-thread folios (ARRAY tag); propose victims from scan FIFO only when >1024 pages, else NOTHING
17	624a8c66	real	645.4	645.4	179	20237	41	dose test: conservative-assist (scan-only victims >1024) rate-limited to 1 of 16 evict calls. Full-rate = 605.6, no-victims = 643.3, noop 63
18	d29bf242	real	620.7	620.7	176	19616	41	dose test 2: conservative-assist scan-only victims (>1024) at 1 of 4 evict calls. Rate 0=643, 1/16=645, 1=606 on pc252 (noop 633/641). Looki
19	bc488f8e	real	643.7	643.7	178	20102	41	replicate 2 of rate-1/16 scan-only-victim hybrid (624a8c66: 645.4) on pc252; noop 633/641. n=2 vs n=2 test of whether near-noop beats noop
20	486071a8	real =noop	645.8	645.8	180	20641	41	final eval: noop run 3 on pc252 (runs 633.2, 641.3) to pin the noop mean against the near-noop hybrid (645.4, 643.7) and no-victims decompos

All attempts on pc257 (captain-ahab)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	1cb59a64	real =get_scan	511.0	511.0	134	10769	40	baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU)
2	6195a85a	real	600.1	600.1	165	17354	41	scan-first FIFO hybrid: only scan-pid-inserted folios tracked (FIFO, skip file last page), rest left to kernel default reclaim; no metadata
3	069942bd	real =noop	642.2	642.2	186	21776	42	calibration: unmodified noop (kernel default reclaim) on my host pc257, to measure baseline + diagnostic probes
4	aff0e4f2	real	626.1	626.1	184	21290	42	floor test: empty struct_ops hooks ATTACHED via real loader (vs noop which never attaches). Measures cost of cache_ext attach itself; noop=6
5	c53d4286	real	592.9	592.9	164	17000	41	structural 1/3 2Q+ghost: scan list first (keep 256), prob list (A1in, promote on 2nd non-scan access), main LRU, LRU-hash ghost of prob evic
6	bd194ffb	real	631.4	631.4	181	20772	41	ablation of c53d4286: identical 2Q+ghost but scan-thread pages NOT special-cased (no scan list, scan accesses count). Tests hypothesis that
7	be6437eb	real	623.3	623.3	184	21770	41	inversion test on bd194ffb (631): scan-thread-inserted folios admitted straight to main LRU instead of probation; GET-inserted still via pro
8	18525923	real	640.9	640.9	177	19924	41	structural 1/3 2Q-CLOCK (on bd194ffb, no scan special-casing): main list becomes freq CLOCK (access freq++ cap 7, no list_move; eviction hal
9	fee0c4a4	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
10	46a06721	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.51s error=benchmark[all] returned 1
11	4d0f5eb0	real	600.6	600.6	159	16006	41	structural 2/3 TinyLFU-2Q (retry 2: hash on scalar host->i_ino, verifier rejected pointer math in sample cb): sketch page history survives e
12	ad91bf2b	real	629.6	629.6	178	20312	41	structural 3/3 TinyLFU-2Q: sketch used for ADMISSION only (insert->main if sketch>=2 or ghost) on top of 2Q-CLOCK 18525923 (641); main back
13	df3b0515	real	628.3	628.3	172	18778	41	tune 2Q-CLOCK 18525923 (641): probation share 1/4 -> 1/10 (S3-FIFO-like small queue), bigger freq-CLOCK main
14	322a0419	real	611.5	611.5	177	19911	40	replication of 2Q-CLOCK 18525923 (641) unchanged, to check whether it is reliably >= noop (642) on pc257 or a lucky draw
15	571edc8a	real =noop	639.0	639.0	185	21617	42	replication of noop calibration 069942bd (642), comment-only change: 2Q-CLOCK replicated 641->612, so spread ~5%; need noop's spread to judg
16	4816c9f3	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
17	5e1449a1	real	613.8	613.8	171	18490	41	2Q-CLOCK-ARRAY retry (rename struct slot -> ahab_slot, clashed with vmlinux.h): same decisions as 2Q-CLOCK 18525923 (mean 626) with direct-m
18	16af2944	real	616.0	616.0	173	19201	41	structural 1/2 junk-first hybrid: we only evict probation (pages never re-accessed after insert: readahead neighbours/one-hit); 2nd access o
19	52469543	real	635.4	635.4	182	21024	42	replication of attached-empty floor aff0e4f2 (626): all 9 of my victim-proposing policies land 593-641 (wall ~40.8s) vs noop 642/639 and att
20	41c629e2	real	621.5	621.5	177	19876	41	final: replication 3 of 2Q-CLOCK 18525923 (641, 612) for a 3-sample mean. Summary of my 20 evals on pc257: noop 642/639, attached-empty 626/

All attempts on pc258 (jack-sparrow)

#	commit	class	score	tput	read MiB	refaults	wall s	design / title
1	3633cdeb	real =get_scan	515.8	515.8	136	11138	40	baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU)
2	8db05066	real	632.0	632.0	166	17497	41	lean scan-first LRU: scan_pids list evicted first (keep youngest 256), general list LRU via list_move on non-scan access, no hash metadata,
3	4ee45189	real	547.4	547.4	143	12524	41	ablation: reference get_scan with ONLY scan-list threshold 5000->256 (and no early return when scan_pages==0). Isolates threshold vs sampled
4	b65fa44f	real	648.9	648.9	163	16831	41	v3: scan-first LRU + probation list. New non-scan folios go to prob_list; first non-scan access promotes to main LRU. Evict order scan(>256)
5	638222b9	real	636.2	636.2	161	16360	41	v3 + SCAN_KEEP 256->2048 (8MiB reserve for unconsumed scan readahead). Tests whether scan-thread speed binds throughput (team sees rbytes po
6	60e63b78	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.90s error=benchmark[all] returned 1
7	25d5e3f8	real =noop	634.1	634.1	176	19502	41	calibration: unmodified noop on MY host pc258 (comment-only change). My v3=649 here; ahab noop on pc257=642. Needed to know whether v3 beats
8	4ed3df5e	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.86s error=benchmark[all] returned 1
9	26392552	real	633.9	633.9	162	16649	41	v5 retry (fix verifier: no ptr arithmetic): v3 decisions, folio_accessed uses direct-mapped ARRAY keyed by hash(ino,index) with exact (ino<<
10	4391835a	real	627.4	627.4	164	17015	41	replication 2 of v3 (b65fa44f, scored 649; comment-only change). ahab showed identical code varies 593->631, so noise ~+-5%; need repeat bef
11	ae41cad6	real	638.0	638.0	162	16497	41	structural 1/3 on cheap-metadata SLRU+ghost (v6): 4 lists scan>prob>main>hot; main=accessed once (the read), hot=2nd access, hot evicted fir
12	c3358345	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.56s error=benchmark[all] returned 1
13	f901bb62	real	633.1	633.1	166	17380	41	structural 2/3 retry (fix verifier: zero-init tag vars). v7 = v6 SLRU+ghost + repeat-scan admission: scan-thread read of a page evicted with
14	7d80fe2d	real	634.9	634.9	160	16249	41	replication 2 of v6 SLRU+ghost array-metadata (ae41cad6, 638). Reviewer goal: per-host mean+-sd for top candidates vs noop. pc258 so far: no
15	b7a7b571	real	629.3	629.3	164	16923	40	structural 3/3 (scan/2nd-level lane) v8 = v6 + exclusive caching: once-read pages go to HEAD of main (MRU-evicted) since LevelDB block cache
16	11a806f6	real	608.7	608.7	182	21358	40	FALSIFICATION (reviewer, deliberately bad): v8 with eviction order inverted to hot > main > prob > scan. Question: does GET victim choice ma
17	c6f2b46c	real =noop	630.3	630.3	176	19659	41	calibration run 2: unmodified noop on pc258 (comment-only). Reviewer conclusion so far: falsification 11a806f6 (GET hot/main evicted FIRST)
18	ecfef48b	real	623.3	623.3	159	16009	41	replication 3 of v6 SLRU+ghost array-metadata (ae41cad6: 638, 635). pc258 means so far: noop 632 (634,630); scan-first family ~635. Falsific
19	1ff6958a	real	639.0	639.0	160	16148	41	replication 3 of v3 (b65fa44f: 649, 627). pc258 means: noop 632 (634,630); v6 632 (638,635,623). Testing whether v3 (no metadata, LRU main)
20	510f3bc0	real	644.1	644.1	162	16485	41	FINAL (20/20): v3 with SCAN_KEEP 256->32 (last untested scan-reserve point; 2048 was neutral). pc258 summary: v3 mean 638.3 (649,627,639) vs

All attempts on pc318 (davy-jones)

#	commit	class	score	tput	read MiB	refaults	walls	design / title
1	3ce2381b	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
2	182386a3	real	584.9	584.9	162	16627	41	scan-aware S3-FIFO v1 retry (fix compile: name clash need_more, ctx param shadows BPF_PROG ctx)
3	86392057	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
4	f32e85c6	real	571.8	571.8	158	15627	41	scan-first + sampled LFU (16 samples) with persistent freq history (LRU hash 200k keyed inode+index restores count on re-add); scan_list evi
5	264e80e8	real	618.6	618.6	161	16444	41	replication: jack-sparrow v3 (b65fa44f) unchanged on my host pc318 — same-host baseline for my composites; checks whether 649 > noop 642 hol
6	9049727e	real	600.0	600.0	160	16178	41	ablation (reviewer): jack v3 + per-folio HASH metadata maintained in added/accessed/evicted/evict-cb but never affecting decisions. Same-hos
7	027fcf58	real	599.9	599.9	159	16101	40	perf-eng: jack v3 + per-CPU array caches in front of scan_pids(tid) and inode_watchlist(ino) hash lookups (revalidate every 256 uses). Motiv
8	c6c5e41e	real	587.1	587.1	156	15408	40	replication run 2 of my LFU+history (f32e85c6, 571.8): ahab showed identical code varies 593->631, so checking whether frequency lane really
9	ab93a6ca	real ✗	crash	-	0	0	4	eval: [FAIL] wall=4.00s error=benchmark[all] returned 1
10	3e850608	real	590.2	590.2	166	17611	41	structural 2/3 (fix verifier: no pointer arithmetic on folio): hash-free CLOCK ref bits, slot = hash(i_ino, index) into 2^24-bit ARRAY bitma
11	a6c7a8aa	real	621.1	621.1	176	19725	41	cross-host replication (reviewer): captain-ahab 2Q-CLOCK 18525923 (640.9 on ahab host) unchanged on my host pc318; my same-host v3 = 618.7
12	5472b2cd	real	618.9	618.9	178	20244	41	composite on ahab 2Q-CLOCK (18525923; 621.1 on my host = my best): ghost LRU 16384 -> 65536 entries (~5x the ~12k-page cache) so long-distan
13	5365201e	real	631.4	631.4	180	20699	42	host calibration retry: unmodified noop (kernel default, never attaches) on my host pc318 — previous attempt 86392057 failed (dropped sys/st
14	82fe0b30	real	615.2	615.2	177	19990	41	ablation (reviewer): ahab 2Q-CLOCK (18525923; my host 621.1/618.9) WITHOUT the last-page-of-file eviction skip that every top policy carries
15	e0f42e0a	real	628.1	628.1	180	20512	41	replication run 2 of noop on pc318 (run 1 = 631.4, 5365201e). Building n>=2 same-host comparison noop vs best cache_ext (2Q-CLOCK 621.1/618.
16	7d097caa	real	599.0	599.0	178	20289	41	structural 1/3 Flag-2Q (overhead lane): NO folio_accessed hook, NO per-folio map. 2Q promotion test = kernel PG_active (set by folio_mark_ac
17	947dc53f	real	605.9	605.9	161	16456	41	replication run 2 of jack v3 on pc318 (run 1 = 618.7, 264e80e8) for n=2 same-host comparison vs noop (631.4/628.1). Also: Flag-2Q 598.97 fal
18	398112c8	real	602.9	602.9	176	19609	41	replication run 3 of ahab 2Q-CLOCK (18525923) on pc318 (runs: 621.1, 618.9[ghost64k]). Same-host means so far: noop 629.8 (n=2), v3 612.3 (n
19	215802ea	real	634.8	634.8	181	20649	42	replication run 3 of noop on pc318 (631.4, 628.1) to balance n=3 same-host comparison: 2Q-CLOCK 621.1/618.9/602.9, v3 618.7/605.9
20	c77b5f03	real	584.8	584.8	159	15852	41	FINAL 20/20 davy-jones: v3 run 3 on pc318. SYNTHESIS (same host pc318): noop 631.4/628.1/634.8 (mean 631.4) > 2Q-CLOCK 621.1/618.9/602.9 (61

pc258 (jack-sparrow): best policy #15 b65fa44f — score 648.9, 648.9 ops/s — overall best

Agent's title: v3: scan-first LRU + probation list. New non-scan folios go to prob_list; first non-scan access promotes to main LRU. Evict order scan(>256) > prob > main. Hypothesis: readahead neighbours (~14KB/IO => ~3 junk pages per GET miss) were polluting LRU MRU end. Also: ablation 4ee45189 showed get_scan threshold fix only 516->547, so sampled-LFU itself loses to LRU (632).

Closest seed by line similarity: mru 64%, sampling 54%, fifo 54%. Lines that differ from mru are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// Lean scan-first LRU (jack-sparrow): two lists, no per-folio hash metadata.
//  - scan_list: folios added by the scan thread (harness scan_pids map). Evicted first,
//    oldest first, keeping only the youngest SCAN_KEEP pages (readahead not yet consumed).
//  - prob_list: new non-scan folios (incl. readahead neighbours). Evicted after scan pages.
//  - gen_list: main LRU; a non-scan access moves the folio here (tail), promoting it from
//    prob_list or scan_list. Unaccessed readahead junk never reaches gen_list.
//  - LevelDB index block (last page of a file) is skipped on eviction.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define SCAN_KEEP 256

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, int);
    __type(value, bool);
    __uint(max_entries, 100);
} scan_pids SEC(".maps");

static u64 scan_list;
static u64 gen_list;
static u64 prob_list;
static s64 scan_added;
static s64 scan_taken;

static inline bool is_scanning_pid(void)
{
    int pid = (int)(bpf_get_current_pid_tgid() & 0xFFFFFFFF);
    return bpf_map_lookup_elem(&scan_pids, &pid) != NULL;
}

static inline bool is_folio_relevant(struct folio *folio)
{
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;
    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline bool is_last_page_in_file(struct folio *folio)
{
    struct address_space *mapping = folio->mapping;
    if (!mapping)
        return false;
    struct inode *inode = mapping->host;
    if (!inode)
        return false;
    unsigned long long file_size = i_size_read(inode);
    if (file_size == 0)
        return false;
    unsigned long long last = (file_size - 1) >> 12;
    return folio_index(folio) == last;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    scan_list = bpf_cache_ext_ds_registry_new_list(memcg);
    gen_list = bpf_cache_ext_ds_registry_new_list(memcg);
    prob_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (scan_list == 0 || gen_list == 0 || prob_list == 0) {
        bpf_printk("cache_ext: init: Failed to create lists\n");
        return -1;
    }
    return 0;
}

static inline bool evictable(struct folio *folio)
{
    if (!folio_test_uptodate(folio) || !folio_test_lru(folio))
        return false;
    if (folio_test_dirty(folio) || folio_test_writeback(folio))
        return false;
    return true;
}

static int scan_evict_cb(int idx, struct cache_ext_list_node *a)
{
    if (!evictable(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    __sync_fetch_and_add(&scan_taken, 1);
    return CACHE_EXT_EVICT_NODE;
}

static int gen_evict_cb(int idx, struct cache_ext_list_node *a)
{
    if (!evictable(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    if (is_last_page_in_file(a->folio))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    s64 scan_len = scan_added - scan_taken;
    if (scan_len > SCAN_KEEP) {
        bpf_cache_ext_list_iterate(memcg, scan_list, scan_evict_cb, eviction_ctx);
        if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
            return;
    }
    bpf_cache_ext_list_iterate(memcg, prob_list, gen_evict_cb, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate(memcg, gen_list, gen_evict_cb, eviction_ctx);
    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict)
        bpf_cache_ext_list_iterate(memcg, scan_list, scan_evict_cb, eviction_ctx);
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio)
{
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid()) {
        if (bpf_cache_ext_list_add_tail(scan_list, folio) == 0)
            __sync_fetch_and_add(&scan_added, 1);
        return;
    }
    bpf_cache_ext_list_add_tail(prob_list, folio);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    if (is_scanning_pid())
        return;
    bpf_cache_ext_list_move(gen_list, folio, true);
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END
```

```c
// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the GET-SCAN policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <signal.h>
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "evo_policy.skel.h"
#include "dir_watcher.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = { { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
                    { "cgroup_path", 'c', "PATH", 0,
                      "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
                    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 } };

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static volatile sig_atomic_t exiting;

static void sig_handler(int signo)
{
    exiting = 1;
}

int main(int argc, char **argv)
{
    int ret = 1;
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    int cgroup_fd = -1;
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    {
        struct sigaction sa;
        memset(&sa, 0, sizeof(sa));
        sigemptyset(&sa.sa_mask);
        sa.sa_handler = sig_handler;
        if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
            perror("Failed to set up signal handling");
            return 1;
        }
    }

    // Parse command line arguments
    struct cmdline_args args = { 0 };
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, &args);

    // Validate arguments
    if (args.watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args.cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    // Does watch_dir exist?
    if (access(args.watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n",
            args.watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    char watch_dir_full_path[PATH_MAX];
    if (realpath(args.watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // TODO: Enable longer length
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    // Open skel
    skel = evo_policy_bpf__open();
    if (skel == NULL) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set watch_dir
    skel->rodata->watch_dir_path_len = strlen(watch_dir_full_path);
    strcpy(skel->rodata->watch_dir_path, watch_dir_full_path);

    // The benchmark harness creates and pins the scan_pids map before any policy loads
    // (and removes the pin afterwards). Reuse it; do not pin our own.
    {
        int scan_pids_fd = bpf_obj_get("/sys/fs/bpf/cache_ext/scan_pids");
        if (scan_pids_fd >= 0) {
            if (bpf_map__reuse_fd(skel->maps.scan_pids, scan_pids_fd))
                fprintf(stderr, "Failed to reuse scan_pids map fd\n");
            close(scan_pids_fd);
        } else {
            fprintf(stderr, "no scan_pids pin found\n");
        }
    }

    // Load programs
    ret = evo_policy_bpf__load(skel);
    if (ret) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    // Initialize inode_watchlist map
    ret = initialize_watch_dir_map(args.watch_dir,
                        bpf_map__fd(skel->maps.inode_watchlist), false);

    // Attach cache_ext_ops to the specific cgroup
    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // Attach probes
    ret = evo_policy_bpf__attach(skel);
```

```
    if (ret) {
        perror("Failed to attach BPF programs");
        goto cleanup;
    }

    printf("evo_policy (GET-SCAN) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc252 (captain-nemo): best policy #80 486071a8 — score 645.8, 645.8 ops/s

Agent's title: final eval: noop run 3 on pc252 (runs 633.2, 641.3) to pin the noop mean against the near-noop hybrid (645.4, 643.7) and no-victims decomposition (643.3)

Closest seed by line similarity: noop 99%, fifo 58%, mru 40%. Lines that differ from noop are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// No-op: registers struct_ops but makes no eviction decisions; vanilla
// kernel reclaim, the reference starting point.
// (jack-sparrow calibration copy on pc258)
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    return 0;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Baseline loader: parses the same args as a real policy and holds the cgroup
// open until SIGINT, but never attaches struct_ops. The kernel runs its
// default page-cache eviction policy on the cgroup, giving a clean "vanilla
// Linux" reference point.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <bpf/bpf.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

struct cmdline_args {
    char *watch_dir;
    uint64_t cgroup_size;
    char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch (ignored by noop)" },
    { "cgroup_size", 's', "SIZE", 0, "Size of the cgroup in bytes" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup" },
    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) { exiting = 1; }

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w': args->watch_dir = arg; break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno) args->cgroup_size = 0;
        break;
    case 'c': args->cgroup_path = arg; break;
    default: return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);
    if (!args->cgroup_path) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }
    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct sigaction sa;
    int cgroup_fd = -1;
    int ret = 1;

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    printf("evo_policy (NOOP baseline) running — kernel default policy. "
           "Press Ctrl+C to exit...\n");
    fflush(stdout);

    while (!exiting)
        sleep(1);
    ret = 0;

    close(cgroup_fd);
    return ret;
}
// EVOLVE-BLOCK-END
// calibration run by captain-nemo on pc252
// noise replicate 2 on pc252
// noop run 3
```

pc257 (captain-ahab): best policy #10 069942bd — score 642.2, 642.2 ops/s

Agent's title: calibration: unmodified noop (kernel default reclaim) on my host pc257, to measure baseline + diagnostic probes

Closest seed by line similarity: noop 100%, fifo 58%, mru 41%. Lines that differ from noop are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// No-op: registers struct_ops but makes no eviction decisions; vanilla
// kernel reclaim, the reference starting point.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    return 0;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Baseline loader: parses the same args as a real policy and holds the cgroup
// open until SIGINT, but never attaches struct_ops. The kernel runs its
// default page-cache eviction policy on the cgroup, giving a clean "vanilla
// Linux" reference point.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <bpf/bpf.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

struct cmdline_args {
    char *watch_dir;
    uint64_t cgroup_size;
    char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch (ignored by noop)" },
    { "cgroup_size", 's', "SIZE", 0, "Size of the cgroup in bytes" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup" },
    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) { exiting = 1; }

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w': args->watch_dir = arg; break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno) args->cgroup_size = 0;
        break;
    case 'c': args->cgroup_path = arg; break;
    default: return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);
    if (!args->cgroup_path) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }
    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct sigaction sa;
    int cgroup_fd = -1;
    int ret = 1;

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    printf("evo_policy (NOOP baseline) running — kernel default policy. "
           "Press Ctrl+C to exit...\n");
    fflush(stdout);

    while (!exiting)
        sleep(1);
    ret = 0;

    close(cgroup_fd);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc318 (davy-jones): best policy #67 215802ea — score 634.8, 634.8 ops/s

Agent's title: replication run 3 of noop on pc318 (631.4, 628.1) to balance n=3 same-host comparison: 2Q-CLOCK 621.1/618.9/602.9, v3 618.7/605.9

Closest seed by line similarity: noop 95%, fifo 58%, mru 40%. Lines that differ from noop are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// No-op: registers struct_ops but makes no eviction decisions; vanilla
// kernel reclaim, the reference starting point. (davy-jones host calibration, run 3)
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    return 0;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Baseline loader: parses the same args as a real policy and holds the cgroup
// open until SIGINT, but never attaches struct_ops.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <bpf/bpf.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

struct cmdline_args {
    char *watch_dir;
    uint64_t cgroup_size;
    char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch (ignored by noop)" },
    { "cgroup_size", 's', "SIZE", 0, "Size of the cgroup in bytes" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup" },
    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) { exiting = 1; }

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w': args->watch_dir = arg; break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno) args->cgroup_size = 0;
        break;
    case 'c': args->cgroup_path = arg; break;
    default: return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);
    if (!args->cgroup_path) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }
    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct sigaction sa;
    int cgroup_fd = -1;

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    printf("evo_policy (NOOP baseline) running\n");
    fflush(stdout);

    while (!exiting)
        sleep(1);

    close(cgroup_fd);
    return 0;
}
// EVOLVE-BLOCK-END
```